

JSON and JQ Fluency

Objectives

1. Read a Management API JSON response and identify the error envelope on sight.
2. Recognize the pagination signal (total > to) and handle it correctly.
3. Read a JQ filter line by line and predict what it will produce.
4. Explain the JSON-out / JSON-in asymmetry and demonstrate the correct transformation.
5. Build a reusable cheat sheet for converting show output into a safe set command.

Lab Environment

Resource	Address	Credentials
SMS	10.1.1.100	admin / Cpwins!1
GW	10.1.1.111	admin / Cpwins!1
Jump server (Windows)	via MobaXterm bookmark	—

The lab is **pre-configured** by the instructor. The SMS API listens on all IPs; the Standard policy package exists with GW as the target.

Pre-Flight

1. Continue from your Lab 1 SSH session to **SMS** (10.1.1.100). If you closed it, reconnect via **MobaXterm**.
2. Confirm jq is installed: `jq --version` should print a version string (1.5 or later).
3. Smoke-test the API:

```
mgmt_cli -r true show api-versions -f json | jq .
```

If any of this fails, raise your hand before continuing.

Task 1: Read the Error Envelope (15 Minutes)

Every Management API error wears the same coat. Learn to recognize it on sight so when a call fails you know which field a script should grep on, and which field tells you what to fix.

1a — Three error responses to read

Below are three error responses captured from real Management API calls. For each one, answer:

1. Which field would a script grep on to decide whether to retry?
2. Which field tells you (the human) what to fix in the call?

Response A — Invalid parameter

```
{
  "code": "generic_err_invalid_parameter",
  "message": "Invalid parameter for [ip-address]",
  "warnings": [],
  "errors": [
    { "message": "ip-address [10.20.30.999] is not a valid IPv4 address" }
  ]
}
```

Response B — Validation failed (publish refused)

```
{
  "code": "err_validation_failed",
  "message": "Validation failed with 0 errors and 2 warnings",
  "warnings": [
    { "message": "Rule 'Cleanup' has Action set to 'Drop' with no log – cannot publish" },
    { "message": "Object 'DB-Server' is unused" }
  ],
  "errors": []
}
```

Response C — Object not found

```
{
  "code": "generic_err_object_not_found",
  "message": "Requested object [old-host] not found",
  "errors": [
    { "message": "object 'old-host' does not exist" }
  ]
}
```

1b — Make a real error happen

In the SMS SSH session, run:

```
mgmt_cli -r true add host name "test-host" ip-address "not-an-ip" -f json
```

Compare what you got back to Responses A, B, and C above. Which envelope does it match?

```
[Expert@SMS:0]# mgmt_cli -r true add host name "test-host" ip-address "not-an-ip" -f json
{
  "code" : "generic_err_invalid_parameter",
  "message" : "Invalid parameter for [ip-address]. Not a valid IPv4 or IPv6 address"
}
Executed command failed. Changes are discarded.
```

1c — The lesson

A script deciding retry vs fail	code — machine-stable string
A human diagnosing what to fix	errors[0].message — names the bad value
Checking whether publish is blocked	warnings — the silent killer (Response B)

Memorize: code is for grep. errors[0].message is for humans. warnings is the silent killer that blocks publish.

Task 2: The Pagination Trap (15 Minutes)

The Management API hands you 50 results at a time by default, with a footer that tells you there are more. Miss that footer and your script's view of the customer's policy is half-empty.

2a — Run a paginated query

```
mgmt_cli -r true show services-tcp -f json | jq '{from, to, total, returned: (.objects | length)}'
```

Expected output shape (numbers may vary with your lab pod):

```
{
  "from": 1,
  "to": 50,
  "total": 219,
  "returned": 50
}
```

`total > to` is your pagination signal. If you ignore it, your script believes there are only 50 TCP Services in the world.

2b — The canonical fix

Two ways to deal with it. First — bump the limit (good if you know an upper bound):

```
mgmt_cli -r true show services-tcp limit 500 -f json | jq '.objects | length'
```

```
[Expert@SMS:0]# mgmt_cli -r true show services-tcp -f json | jq '{from, to, total, returned: (.objects | length)}'
{
  "from": 1,
  "to": 50,
  "total": 219,
  "returned": 50
}
[Expert@SMS:0]# mgmt_cli -r true show services-tcp limit 500 -f json | jq '.objects | length'
219
[Expert@SMS:0]#
```

Second — loop with offset (correct for any size):

```
offset=0; limit=50; all="[]"
while ;; do
  page=$(mgmt_cli -r true show services-tcp offset $offset limit $limit -f json)
  all=$(jq -s '.[0] + .[1].objects' <(echo "$all") <(echo "$page"))
  to=$(echo "$page" | jq .to); total=$(echo "$page" | jq .total)
  [ "$to" -ge "$total" ] && break
  offset=$((to))
done
echo "$all" | jq 'length'
```

```
[Expert@SMS:0]# offset=0; limit=50; all="[]"
all=$(jq -s '.[0] + .[1].objects' <(echo "$all") <(echo "$page"))
to=$(echo "$page" | jq .to); total=$(echo "$page" | jq .total)
[ "$to" -ge "$total" ] && break
offset=$((to))
done
echo "$all" | jq 'length'
[Expert@SMS:0]# while ;; do
> page=$(mgmt_cli -r true show services-tcp offset $offset limit $limit -f json)
> all=$(jq -s '.[0] + .[1].objects' <(echo "$all") <(echo "$page"))
> to=$(echo "$page" | jq .to); total=$(echo "$page" | jq .total)
> [ "$to" -ge "$total" ] && break
> offset=$((to))
> done
[Expert@SMS:0]# echo "$all" | jq 'length'
219
[Expert@SMS:0]#
```

Run both versions. Confirm the count matches the total reported by the API.

2c — Edge case: zero results

Some filters return nothing. The response looks different — different enough to break the loop in 2b. Try it:

```
mgmt_cli -r true show services-tcp filter "name=^impossible-prefix-12345" -f  
json | jq '{from, to, total}'
```

Expected output:

```
{  
  "from": null,  
  "to": null,  
  "total": 0  
}
```

Trace the loop in 2b against this response. Specifically:

6. After `to=$(echo "$page" | jq .to)`, what is the value of `$to`? (Hint: it is not a number.)
7. What does bash do with `[ "$to" -ge "$total" ]` when `$to` is null?
8. Does the loop terminate, error out, or run forever?

Answer. `$to` is the literal string `null`. Bash's `-ge` requires integers, so the test errors out. Under `set -e` the script dies; without it, the test silently evaluates false, `break` never fires, and the loop runs forever — hammering the API for empty pages.

Fix. Add an explicit guard before the integer comparison:

```
[ "$total" -eq 0 ] && break  
[ "$to" -ge "$total" ] && break
```

The lesson. Every paginated API response has an "empty" shape that is structurally different from the populated shape. Loops that assume one set of field types break on the other.

Task 3: Read JQ Filters Fluently (20 Minutes)

You will not memorize every JQ operator. You will, however, read JQ filters fluently enough to know what one does before you pipe customer data through it.

3a — Pull gateway data into a file

```
mgmt_cli -r true show simple-gateway name "GW" -f json > /tmp/gw.json
```

3b — Study the filter before running it

Below is a JQ filter intended to print each interface as "name: ipv4-address". Read it before running it.

```
jq -r '.interfaces[] | "\(.name): \(.ipv4-address)"' /tmp/gw.json
```

Answer these before you run it:

3. What does `.interfaces[]` do?
4. What does the `|` between expressions mean inside JQ?
5. Why is "ipv4-address" wrapped in quotes when `.name` is not?
6. What does the `-r` flag on `jq` change about the output?

3c — Run it and confirm

Run the filter from 3b. Expected output shape:

```
eth1: 10.1.2.111
eth2: 10.1.3.111
eth0: 10.1.1.111
eth4: 198.51.100.111
eth3: 203.0.113.111
```

```
[Expert@SMS:0]# mgmt_cli -r true show simple-gateway name "GW" -f json > /tmp/gw.json
[Expert@SMS:0]# jq -r '.interfaces[] | "\(.name): \(.ipv4-address)"' /tmp/gw.json
eth1: 10.1.2.111
eth2: 10.1.3.111
eth0: 10.1.1.111
eth4: 198.51.100.111
eth3: 203.0.113.111
```

3d — The lesson

Answers to 3b: (1) emits each interface as a separate value; (2) inside JQ, a pipe feeds one value through the next expression; (3) `ipv4-address` contains a hyphen which JQ would otherwise parse as subtraction, so quoting forces a literal key; (4) `-r` outputs raw strings (no surrounding quotes).

Task 4: The JSON Symmetry Pain (15 Minutes)

`show <object>` returns a rich JSON document. `set <object>` will not accept that document as input. Every `show → set` operation is a transformation, not a round-trip. This is the #1 API pain point in customer feedback.

4a — Capture a show host response

First, create a host to inspect (if it doesn't already exist), then capture its `show host` JSON:

```
mgmt_cli -r true add host name "DB-Server" ip-address 10.1.1.50 color blue
mgmt_cli -r true show host name "DB-Server" -f json > /tmp/host.json
cat /tmp/host.json | jq .
```

The response includes fields like:

```
[Expert@SMS:0]# mgmt_cli -r true show host name "DB-Server" -f json > /tmp/host.json
[Expert@SMS:0]# cat /tmp/host.json | jq .
{
  "uid": "6cf48ba6-82fd-4ca0-b56c-b33dc3bf466d",
  "name": "DB-Server",
  "type": "host",
  "domain": {
    "uid": "41e821a0-3720-11e3-aa6e-0800200c9fde",
    "name": "SMC User",
    "domain-type": "domain"
  },
  "ipv4-address": "10.1.1.50",
  "interfaces": [],
  "nat-settings": {
    "auto-rule": false
  },
  "groups": [],
  "comments": "",
  "color": "blue",
  "icon": "Objects/host",
  "tags": [],
  "meta-info": {
    "lock": "unlocked",
    "validation-state": "ok",
    "last-modify-time": {
      "posix": 1780957691558,
      "iso-8601": "2026-06-08T18:28-0400"
    },
    "last-modifier": "WEB_API",
    "creation-time": {
      "posix": 1780957691558,
      "iso-8601": "2026-06-08T18:28-0400"
    },
    "creator": "WEB_API"
  },
  "read-only": false,
  "available-actions": {
```

4b — Read it. Which fields would break a set host call?

If you piped this entire document back into `set host`, which fields would the API reject?

uid	Identifier, not a mutable field. Use name OR uid as the target, not as a payload field.
meta-info	Read-only metadata (creator, timestamps).

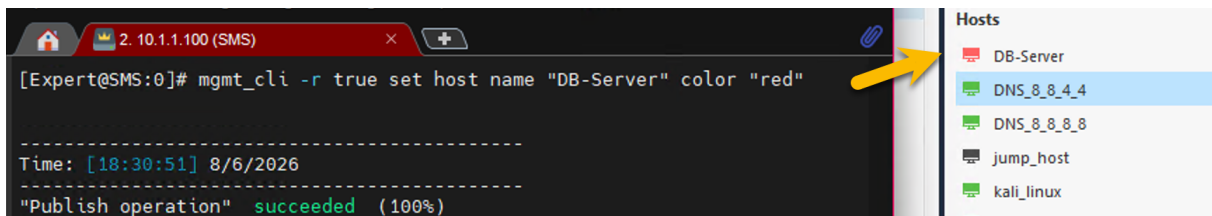
type	Immutable. You don't turn a host into a network with set.
tags (as objects)	set expects tag names (strings), not nested objects with uid/type.
groups	Derived membership, not a settable field on the host itself.

4c — The canonical set command

To change ONLY the color, send a minimal payload that identifies the target and lists the one field to change. No metadata. No echoing the show document.

```
mgmt_cli -r true set host name "DB-Server" color "red"
```

Three tokens. Target by name, one field changed. Confirm in SmartConsole that the host icon changed color.



4d — The wrong way (do not run)

For illustration only. Don't pipe the show document back as a set payload — it fails immediately:

```
# Anti-pattern – do NOT run this. Shown for illustration.
cat /tmp/host.json | mgmt_cli -r true set host
```

Result: err_validation_failed or generic_err_invalid_parameter — the API rejects uid, meta-info, and the rest.

4e — The lesson

Every show → set operation is a transformation, not a round-trip. The transformation rule is simple: keep name (or uid) as the target, drop everything else, add only the fields you want to change. The Checkpoint task below makes this rule reusable.

Checkpoint (10 Minutes)

Write a one-page cheat sheet (or a short shell function) that, given any `show <object>` JSON file and a single field to change, produces the matching `mgmt_cli set <object>` command. The cheat sheet must:

1. Use the object's name (not `uid`) as the target.
2. Change exactly one named field.
3. Never include `uid`, `meta-info`, `type`, `groups`, or `tags-as-objects`.
4. Test it on `/tmp/host.json` from Task 4, changing `color` to `green`. Hand the cheat sheet (or function code) to the instructor.